Structure

- Structure is basically a user-defined data type that can store related information (even of different data types) together.
- Collection of variables under a single name.

Declaration structure type

- A structure is declared using the keyword `struct` followed by the structure name.
- All the variables of a structure are declared within the structure.

```
struct struct-name
{
    data-type var-name;
    data-type var-name;
    ...
};
```

- For example

```
struct student {
    int rollno;
    char name[255];
    float cgpa;
};
```

- Each variable name declared within a structure is called a ***member of the structure***.
- The structure declaration, however, does not allocate any memory or consume storage space.
- The structure declaration, gives a template that conveys to the C compiler how the structure would be laid out in the memory and also gives the details of member names.

Declaring structure variable

- Memory is only allocated when we declare a variable of the structure.

- There are mainly two ways of declaring a structure variable:
  1. `struct struct-name struct-var;`
     - `struct struct-name` is a datatype
     - `struct-var` is a variable
     - For example: `struct student s;`
  2. At the time of structure declaration

```
struct struct-name
{
    data-type var-name;
    data-type var-name;
    ...
} struct-var-1, struct-var-2;
```

```
struct student {
    int rollno;
    char name[255];
    float cgpa;
} s1, s2;
```

- Structure type and variable declaration can be either local or global depending on their placement in the code.

Accessing Members

- The structure member variable is generally accessed using a '.' (dot) operator.
- Syntax -> `struct_var.member_name`
- For example

```
// --snip--
    s1.rollno = 42;
    strcpy(s1.name, "Abanggi Momin");
    s1.cgpa = 8.2;
```

Initialization of structures

- Initializing a structure means assigning some constants to the members of the structure when structure variable is created.
- Implicitly C initializes the structure
  - For `int` and `float` members, the values are initialised to 0 and `char` and string members are initialised to `'\0'` by default.
- Syntax 1

```
struct struct-name
{
    data-type var-name;
    data-type var-name;
    ...
} struct-var = {constant1, constant2, ...};
```

- Syntax 2

```
struct struct-name
{
    data-type var-name;
    data-type var-name;
    ...
};
struct struct-name struct-var = {constant1, constant2,
...};
```

- When all the members of a structure are not initialised, it is called ***partial initialization***.

`typedef` Declarations

- `typedef` keyword enables the programmer to create a new data type name by using an existing data type.
- `typedef` allows us to give an alternate name to a known data type.
- Syntax -> `typedef existing-dtype new-dtype`
- For example

```
#include <stdio.h>
```

```c
int main()
{
    typedef int INTEGER;

    int x = 3;
    INTEGER y = 5;

    printf("y=%d,x=%d", y, x);     // y=5,x=3
    return 0;
}
```

- Using `typedef` with structure

```c
// --snip--
typedef struct {
    int rollno;
    char name[255];
    float cgpa;
} student;
```

```c
// --snip--
student s1, s2;
```

Array of structures

- We can define an array to store multiple structure variables as a unit
- For example – Storing students of a particular class, Storing movies in your library etc.
- **Syntax**
  - During structure declaration

```c
struct struct_name
{
// members
} struct_name struct_var[index];
```

```c
// usage example
struct student {
    int rollno;
```

```
    char name[255];
    float cgpa;
} studs[10];
```

- After structure declaration

```
struct struct_name struct_var[index];
```

```
// usage example
struct student studs[10];
```

Structure & Functions

- We can pass structures as arguments to functions as with our normal variables
- A structure can be passed to functions in three ways
  1. Passing individual members
  2. Passing entire structure
  3. Passing the address of structure

1. Passing individual members

- We simply refer to the member to be passed using '`.`' (dot) operator in the function call
- The called function has no way of knowing if the argument is from an ordinary variable or structure variable
- usage example

```
#include <stdio.h>

typedef struct {
    int rollno;
    char name[255];
    float cgpa;
} Student;

void displayCGPAPerc(float);

int main()
{
```

```
    Student s = {12, "John", 8.2};

    displayCGPAPerc(s.cgpa);

    return 0;
}

void displayCGPAPerc(float cgpa) {
    printf("Your percentage is %.2f%%", cgpa*10.0);
}
```

2. Passing entire structure

- When a structure is passed as an argument, it is passed using the call by value method, i.e., a copy of each member of the structure is made.
- The called function must follow the given syntax `return_type func_name(struct_name struct_var);`
- usage example

```
#include <stdio.h>

typedef struct
{
    int rollno;
    char name[255];
    float cgpa;
} Student;

void displayStudentInformation(Student);

int main()
{
    Student s = {12, "John", 8.2};

    displayStudentInformation(s);

    return 0;
}
```

```
void displayStudentInformation(Student stu)
{
    printf("Student rollno: %d\n", stu.rollno);
    printf("Student name: %s\n", stu.name);
    printf("Student cgpa: %.2f", stu.cgpa);
}
```

3. Passing the address of structure

- Passing large structures to functions using call by value is very inefficient. Hence it is preferred to pass the structure's address instead
- A pointer to a structure is never itself a structure, but a variable that holds the address of a structure
- The called function must follow the general syntax
  `return_type func_name(struct_name *struct_ptr)`
- The function call must follow
  `func_name(&struct_var)`
- To access the individual members from the structure we can either follow
  `(*struct_ptr).member`
  or
  `struct_ptr->member`
  - This -> operator is called "*Pointing to*" operator
- usage example

```
#include <stdio.h>

typedef struct
{
    int rollno;
    char name[255];
    float cgpa;
} Student;

void collectStudentInformation(Student *);
void displayStudentInformation(Student);
```

```c
int main()
{
    Student s;

    collectStudentInformation(&s);
    printf("\n[New Student]\n");
    displayStudentInformation(s);

    return 0;
}

void displayStudentInformation(Student stu)
{
    printf("Student rollno: %d\n", stu.rollno);
    printf("Student name: %s\n", stu.name);
    printf("Student cgpa: %.2f", stu.cgpa);
}

void collectStudentInformation(Student *stu)
{
    printf("Enter student rollno: ");
    scanf("%d", &stu->rollno);

    printf("Enter student name: ");
    scanf("%s", stu->name);

    printf("Enter student cgpa: ");
    scanf("%f", &stu->cgpa);
}
```

Copying and comparing structures

Copying

- We can assign a structure to another structure of the same type
- usage example

```c
struct student stud1 = {12, "John", 8.2};
struct student stud2;
```

```
stud2 = stud1;
```

- above code sets the values of `stud2` members from the values of `stud1` members

Comparing

- C doesn't allow the comparison of one structure with another
- However individual structure members can be compared
- example

```
#include <stdio.h>
#include <string.h>

typedef struct {
    int rollno;
    char name[255];
    float cgpa;
} Student;

/*
*   compares two strings and returns
*       0 -> if equal
*       1 -> otherwise
*/
int compareStudents(Student, Student);

int main()
{
    Student s1 = {12, "John", 8.2};
    Student s2 = s1;

    if (compareStudents(s1, s2) == 0)
        printf("They are the same two students");
    else
        printf("They are two different students");

    return 0;
```

```
}

int compareStudents(Student s1, Student s2)
{
    if (s1.rollno != s2.rollno)
        return 1;

    if (strcmp(s1.name, s2.name) != 0)
        return 1;

    if (s1.cgpa != s2.cgpa)
        return 1;

    return 0;
}
```

Self-referential structures

- Self-referential structures are structures that contain a reference to the data of its same type
- I.e., it has a member which is a pointer to a structure of the same type
- usage example

```
struct node
{
    int val;
    struct node *next;
};
```

- Self-referential structure is the foundation for data structures such as linked lists, trees and graphs

Unions

- Similar to structures, a union is a collection of variables of different data types
- The only difference being, in case of unions, information can be stored in only one field at any one time

- When a new value is assigned to a field, the existing data is replaced with the new data
- Unions are used to save memory. They are useful of applications that involve multiple members, where values need not be assigned to all the members at any one time

Declaring a union

- The syntax is similar to declaring a structure with the only difference being the use of keyword `union` instead of `struct`
- **Syntax**

```
union union-name
{
    data_type var-name;
    data_type var-name;
    ...
};
```

- The size of a union is the size of its largest field such that sufficient number of bytes must be reserved to store the largest sized field

Initializing a union

- example

```
#include <stdio.h>

union Point
{
    int x;
    int y;
};

int main()
{
    union Point p;

    // p = {3, 2}; -> illegal, in case of unions
```

```
    p.x = 3;
    p.y = 2;

    printf("co-ordinates: %d, %d\n", p.x, p.y);
    // co-ordinates: 2, 2

    return 0;
}
```

Union of structures

- A union can have structures as its members, in case of which, any one of the structure will be occupying the space in memory at any given time
- How much memory is allocated to the union depends entirely on it's largest structure
- usage example

```
#include <stdio.h>

typedef struct
{
    float radius;
} Circle;

typedef struct
{
    float length;
    float breadth;
} Rectangle;

typedef union
{
    Circle    cir;
    Rectangle rect;
} Shape;

void displayAllDimensions(Shape);
```

```
int main()
{
    Shape s;

    s.cir.radius = 2.0;
    displayAllDimensions(s);

    s.rect.length = 5.0;
    displayAllDimensions(s);

    s.rect.breadth = 3.0;
    displayAllDimensions(s);

    return 0;
}

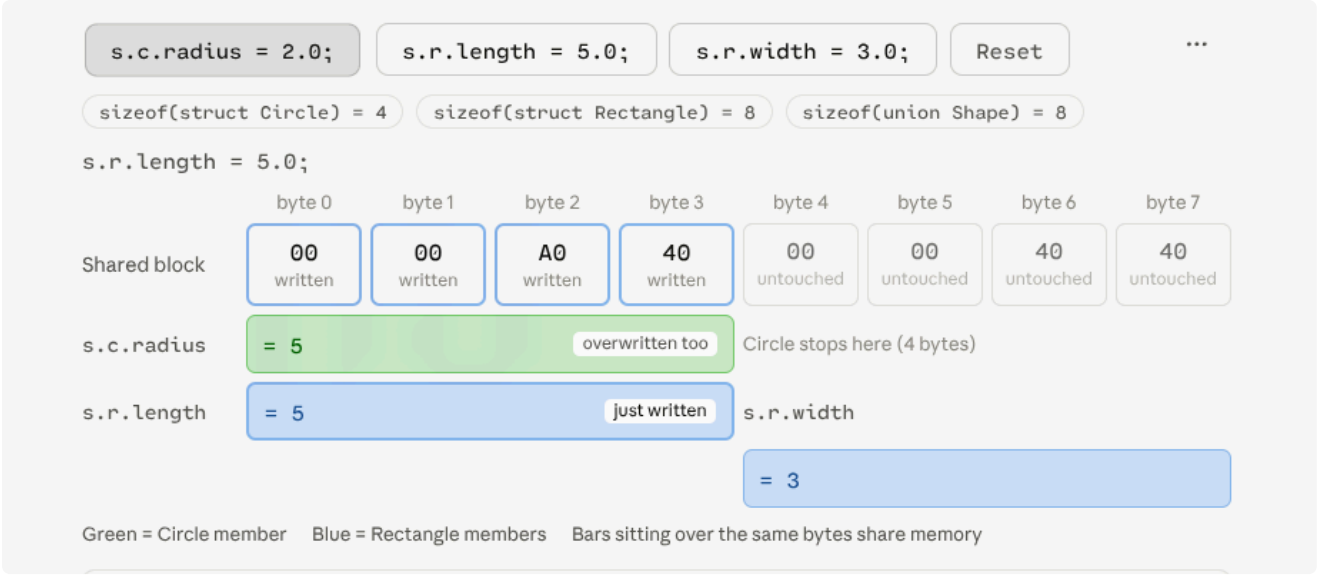
void displayAllDimensions(Shape s)
{
    printf("\nCirlce radius = %.2f\n", s.cir.radius);
    printf("Rectange length = %.2f\n", s.rect.length);
    printf("Rectange breadth = %.2f\n", s.rect.breadth);
};
```

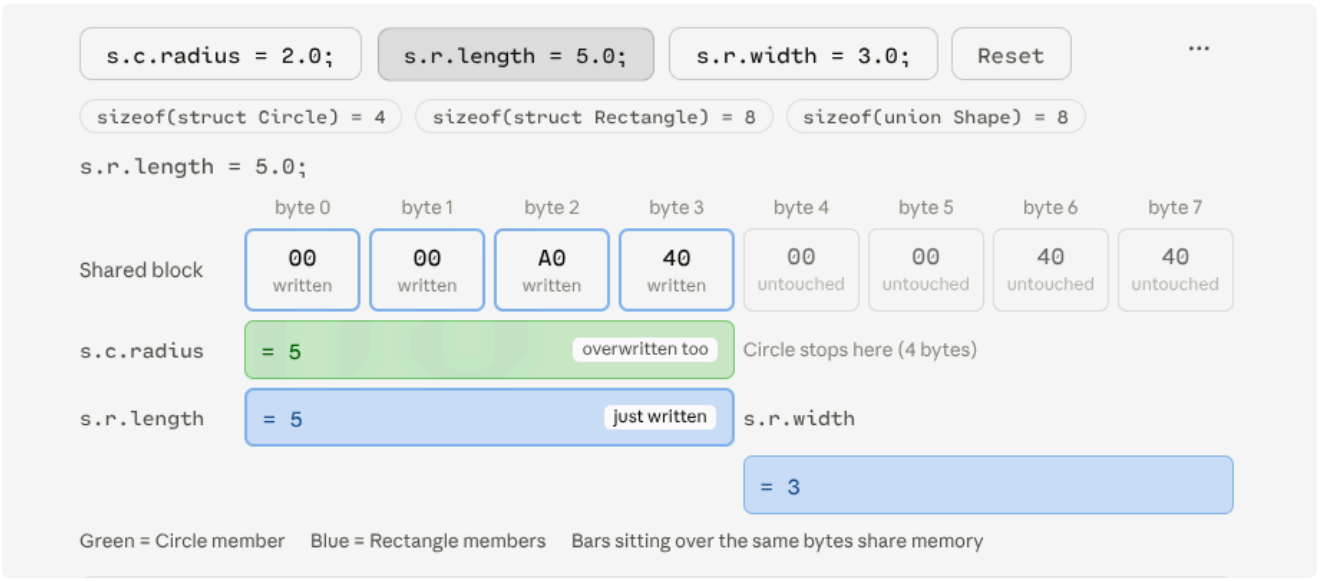
- output

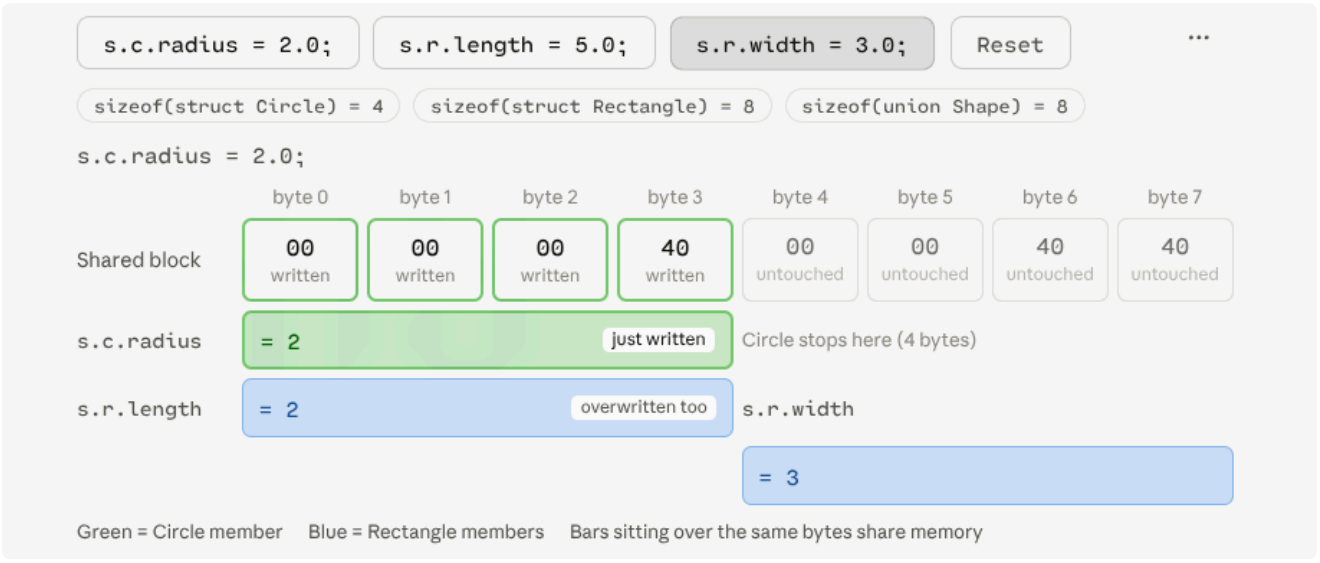
```
Cirlce radius = 2.00
Rectange length = 2.00
Rectange breadth = 0.00

Cirlce radius = 5.00
Rectange length = 5.00
Rectange breadth = 0.00

Cirlce radius = 5.00
Rectange length = 5.00
Rectange breadth = 3.00
```







Enumeration

- An enumeration is a user-defined data type that consists of a set of named integer constants.
- It lets us give meaningful names to integer values, instead of using unexplained numbers (magic numbers) in code.

- Enumeration is declared using the keyword `enum` followed by the enumeration name.
- All the constants of an enumeration are declared within the braces.
- Each constant name declared within an enumeration is called an ***enumerator*** (or ***enumeration constant***).
- **Syntax**

```
enum enum-name
{
    CONSTANT1,
    CONSTANT2,
    ...
    CONSTANTn
};
```

- Just like structure, the enumeration declaration, does not allocate any memory. It only gives a template to the C compiler to convey the new type name and the name of its constants
- By convention, enumeration constants are written in UPPERCASE.
- **Example**

```
enum day {
    SUN,
    MON,
    TUE,
    WED,
    THU,
    FRI,
    SAT
};
```

- To create a variable of a enumeration type we can use the following syntax

```
/*
    Declaring
*/
enum enum-name
```

```
{
    CONSTANT1,
    CONSTANT2,
    ...
    CONSTANTn
} var-name;
// or
enum enum-name var-name;

/*
    Initializing
*/
enum enum-name
{
    CONSTANT1,
    CONSTANT2,
    ...
    CONSTANTn
} var-name = CONSTANT;
// or
enum enum-name var-name = CONSTANT;
```

- **Example**

```
enum day today = WED;
```

- By default, the first constant has the value `0`, and each next constant is one more than the previous one.

Custom enumeration constants

- We can assign our own integer values to the constants using `=`
- A constant without a value continues counting from the previous constant
- **Example**

```
enum http_status
{
    OK = 200,
```

```
    FORBIDDEN = 403,
    NOT_FOUND,
    SERVER_ERROR = 500
};
```

- `NOT_FOUND` will be assigned the constant `404`

`typedef` Declaration of enums

- Syntax

```
typedef enum
{
    CONSTANT1,
    CONSTANT2,
    ...
    CONSTANTn
} enum-name;
```

- Example

```
typedef enum {
    SUN,
    MON,
    TUE,
    WED,
    THU,
    FRI,
    SAT
} day;
```

Using enums with switch case

- **Example**

```
typedef enum
{
    RED,
    YELLOW,
```

```
    GREEN
} light;

int main()
{
    light current = YELLOW;

    switch(current)
    {
        case RED:
            printf("Stop");
            break;
        case YELLOW:
            printf("Wait");
            break;
        case GREEN:
            printf("Go");
            break;
    }

    return 0;
}
```

- Output

```
Wait
```